DESIGN & ANALYSIS
OF ALGORITHMS

Asymptotic Notations

A clear, exam-ready guide to growth rates, upper bounds, lower bounds, tight bounds, and their practical meaning.

Topics covered

01 · Introduction & Purpose
02 · Big-O — Upper Bound
03 · Big-Ω — Lower Bound
04 · Big-Θ — Tight Bound
05 · Important Distinction: Bounds vs. Exact Growth

MCA • DAA Notes
Designed as a concise conceptual reference with definitions, examples and visual intuition.

Contents

A quick map of the chapter

No.	Topic	Focus
01	Introduction to Asymptotic Notations	Why growth rates matter; O, Ω and Θ.
02	Big-O Notation (O) — Upper Bound	Formal definition, intuition and examples.
03	Big-Ω Notation (Ω) — Lower Bound	Formal definition, intuition and examples.
04	Big-Θ Notation (Θ) — Average / Tight Bound	Two-sided bound and tight asymptotic growth.
05	Important Distinction: Bounds vs. Exact Growth	Why O(f(n)) does not mean exact equality.
06	Quick Comparison & Exam Checklist	One-page revision summary.

Core idea

Asymptotic analysis studies how the running time or space requirement of an algorithm grows as the input size n becomes large. Constants and lower-order terms are deliberately ignored because long-run growth is the main concern.

01 · Introduction to Asymptotic Notations

Origin & Purpose

- Asymptotic notation provides mathematical tools for describing the growth rate of an algorithm's running time or space requirement.
- It focuses on behaviour for large input size n, rather than machine-dependent execution time.
- It lets us compare algorithms by their growth rates and ignore constants and lower-order terms.

O(g(n))	Ω(g(n))	Θ(g(n))
Upper bound	Lower bound	Tight bound
f(n) does not grow faster than a constant multiple of g(n), eventually.	f(n) grows at least as fast as a constant multiple of g(n), eventually.	f(n) and g(n) have the same asymptotic order of growth.

Why not use exact running time?

- Exact time depends on hardware, programming language, compiler, operating system and implementation details.
- Asymptotic notation gives a machine-independent description of the algorithm's growth.

Example: $3n^2 + 7n + 10 \rightarrow \Theta(n^2)$

Remember

When n becomes very large, the highest-order term dominates. That is why $3n^2 + 7n + 10$ has the same asymptotic growth as n^2.

02 · Big-O Notation (O) — Upper Bound

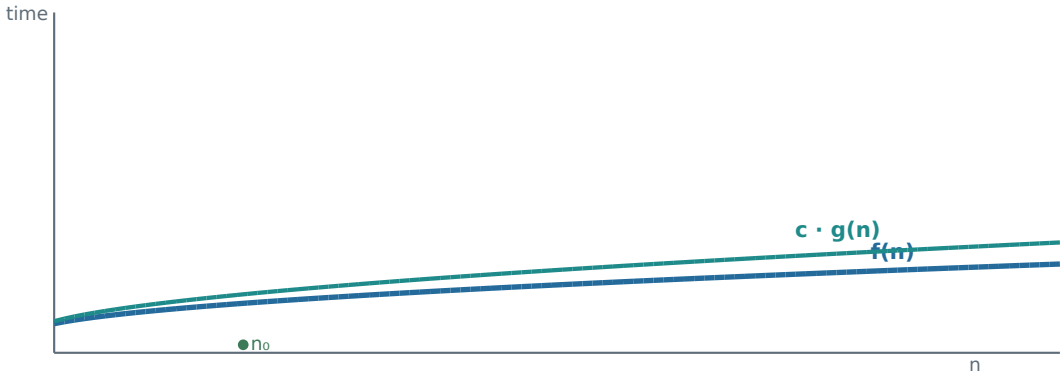
Definition

Big-O notation gives an asymptotic upper bound. It tells us that beyond some input size, f(n) is no more than a constant multiple of g(n).

$f(n) = O(g(n))$ if $\exists\ c > 0$ and $n_0 > 0$ such that $0 \le f(n) \le c \cdot g(n)$ for all $n \ge n_0$

Intuition

- Think of g(n) as a ceiling that eventually stays above f(n), after multiplying g(n) by a suitable constant c.
- Big-O is commonly used to express an algorithm's worst-case growth, although mathematically O itself is a bound and is not synonymous with "worst case."



For sufficiently large n, f(n) stays at or below a constant multiple of g(n).

Worked example

Let $f(n) = 3n^2 + 2n + 1$. For $n \ge 1$, we have $2n \le 2n^2$ and $1 \le n^2$. Therefore:

$f(n) = 3n^2 + 2n + 1 \le 6n^2$ for $n \ge 1$

Choose $c = 6$ and $n_0 = 1$. Hence $f(n) = O(n^2)$.

> **Key point**
>
> The constants c and n_0 do not need to be unique. If one valid pair exists, the Big-O relationship holds.

03 · Big-Ω Notation (Ω) — Lower Bound

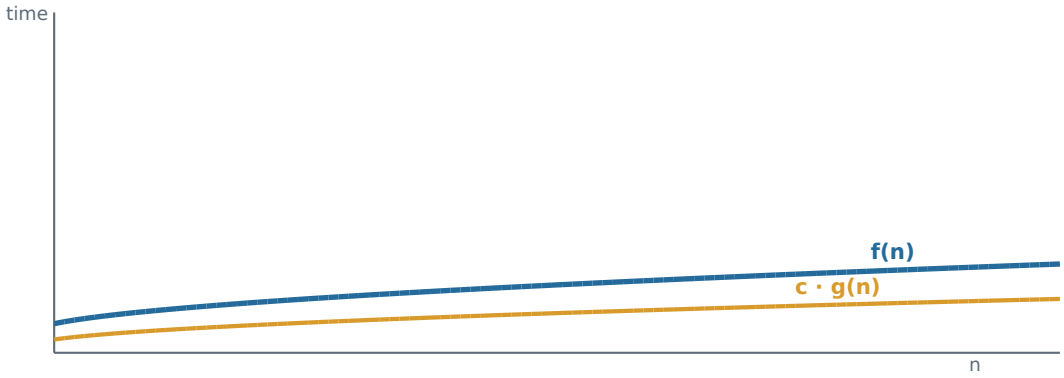
Definition

Big-Ω notation gives an asymptotic lower bound. It says that, from some point onward, f(n) is at least a constant multiple of g(n).

$f(n) = \Omega(g(n))$ if $\exists\ c > 0$ and $n_0 > 0$ such that $0 \le c \cdot g(n) \le f(n)$ for all $n \ge n_0$

Intuition

- Think of c·g(n) as a floor that eventually remains below f(n).
- Ω is useful when establishing a guaranteed minimum growth or lower limit on the resources required.



For sufficiently large n, f(n) stays at or above a constant multiple of g(n).

Worked example

For $f(n) = 3n^2 + 2n + 1$, clearly $f(n) \ge 3n^2$ for every $n \ge 1$. Thus:

$$3n^2 \le f(n) \text{ for } n \ge 1$$

Choose $c = 3$ and $n_0 = 1$. Hence $f(n) = \Omega(n^2)$.

Exam language

If f(n) = Ω(g(n)), then f(n) grows at least as fast as g(n), up to a positive constant factor, for sufficiently large n.

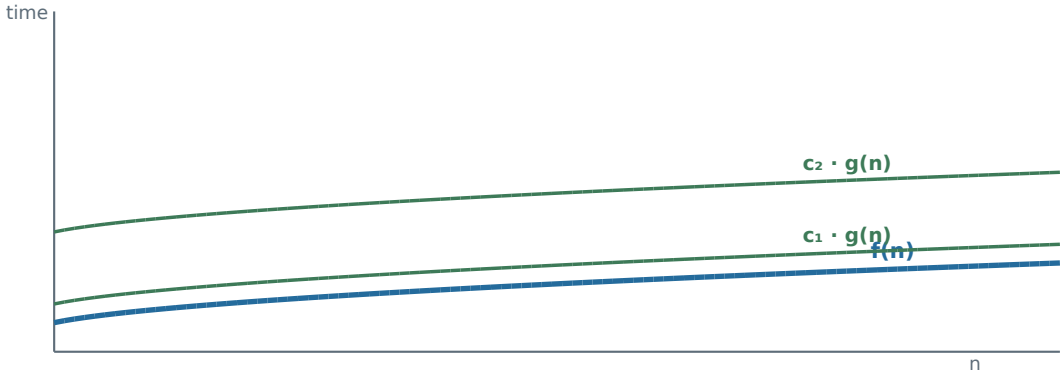
04 · Big-Θ Notation (Θ) — Tight Bound

Definition

Big-Θ gives a tight asymptotic bound. It combines an upper bound and a lower bound so that f(n) is eventually trapped between two constant multiples of g(n).

$f(n) = \Theta(g(n))$ if $\exists\ c_1, c_2 > 0$ and $n_0 > 0$ such that $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$ for all $n \ge n_0$

Visual intuition



Eventually, f(n) is trapped between two constant multiples of g(n).

Worked example

Let $f(n) = 4n^2 + 3n + 2$. For $n \ge 1$:

$$4n^2 \le 4n^2 + 3n + 2 \le 9n^2$$

Therefore, with $c_1 = 4$, $c_2 = 9$ and $n_0 = 1$, we get $f(n) = \Theta(n^2)$.

> **Relationship**
>
> If $f(n) = \Theta(g(n))$, then both $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$. A tight bound is therefore stronger information than an upper bound alone.

05 · Important Distinction: Bounds vs. Exact Growth

Common misconception

- O(g(n)) does not mean f(n) is exactly g(n). It only says g(n) is an asymptotic upper bound.
- An algorithm can belong to many Big-O classes. For example, if f(n) = n, then f(n) is O(n), O(n^2), O(n^3), and so on.
- Among these, O(n) is the more informative upper bound because it is tighter.

Bound vs. tight description

Notation	Meaning	Example for f(n)=n^2
O(g(n))	Upper bound: f(n) eventually stays at or below c·g(n).	O(n^2), O(n^3), O(n^4)… all are true.
Ω(g(n))	Lower bound: f(n) eventually stays at or above c·g(n).	Ω(n^2), but also Ω(n) is true.
Θ(g(n))	Tight bound: both upper and lower bounds hold.	Θ(n^2) is the tight asymptotic order.

Very important

Saying “f(n) = O(g(n))” is not the same as saying “f(n) = Θ(g(n))”. O allows looser upper bounds; Θ identifies the same asymptotic order of growth.

A useful mental model

O → ceiling / upper limit • Ω → floor / lower limit • Θ → both ceiling and floor

Growth-rate examples

Growth	Typical class	Relative behaviour
1	Θ(1)	Constant
log n	Θ(log n)	Very slow
n	Θ(n)	Linear
n log n	Θ(n log n)	Slightly above linear
n^2	Θ(n^2)	Quadratic
2^n	Θ(2^n)	Exponential

06 · Quick Comparison & Exam Checklist

One-glance comparison

06 · Quick Comparison & Exam Checklist

One-glance comparison

06 · Quick Comparison & Exam Checklist

One-glance comparison

Notation	Bound	Formal idea	Memory cue
O(g(n))	Upper	$f(n) \le c \cdot g(n)$ eventually	Ceiling
Ω(g(n))	Lower	$c \cdot g(n) \le f(n)$ eventually	Floor
Θ(g(n))	Tight	$c_1 g(n) \le f(n) \le c_2 g(n)$ eventually	Sandwich

How to prove a Big-O / Ω / Θ statement

- Step 1: Write the formal inequality you need.
- Step 2: Choose suitable constants (c, or c_1 and c_2).
- Step 3: Choose a threshold n_0 from which the inequality is guaranteed.
- Step 4: Verify the inequality for all $n \ge n_0$.

> **Fast exam memory**
>
> O = upper bound • Ω = lower bound • Θ = tight bound. If you can prove both O and Ω for the same g(n), you have Θ(g(n)).

Mini practice

Classify the following as tightly as possible: (a) $5n + 12$, (b) $7n^2 + 4n + 9$, (c) $3n^3 + n$, (d) $2^n + n^2$.

Answers: (a) $\Theta(n)$ (b) $\Theta(n^2)$ (c) $\Theta(n^3)$ (d) $\Theta(2^n)$

End of Notes

Asymptotic notation is fundamentally about growth, not exact execution time. Learn the three symbols as a set: O → upper, Ω → lower, Θ → tight.